Technical Documentation

Multi-Metal Analytical Transport Model

ADE_Analytical_Multimetal.py — Complete Technical Reference

Author	W.M. Kanchana D. Wijeratna
Affiliation	Hydro-Environment System Laboratory, Dept. of Civil Engineering, Tohoku University
Supervisor	Prof. So Kazama
Version	2.0 (Analytical ADE + Multi-metal)
Language	Python 3 | PCRaster framework | NumPy
Purpose	Forward pollutant transport model for inverse source identification

1. Overview and purpose

This model simulates the downstream transport of heavy metal pollutants through a river basin. It is the forward component of a larger inverse modelling framework aimed at identifying pollutant sources in complex river networks from downstream observations. Two major advances over the original finite-volume upwind model are incorporated: (1) an analytical solution of the advection–diffusion equation replaces the first-order upwind finite-difference scheme, eliminating numerical diffusion; and (2) each metal species is transported independently, allowing multi-metal concentration maps to be produced from a single run using chemical signature data.

1.1 Design philosophy

The model combines two complementary tools. PCRaster handles everything related to river network topology — the Local Drain Direction map routes mass through confluences and bifurcations exactly, using the upstream() accumulation operator. The analytical ADE solution handles the physics of transport within each channel segment — advection, physical dispersion, and optional first-order decay — without the artificial smearing that the upwind scheme introduces. This operator-splitting approach means each tool does what it is best at.

1.2 What the model produces

For each metal species and each timestep, the model writes a PCRaster map of mass per unit length M [mg/m] and optionally concentration C [mg/m³]. At the end of the run it produces a mass balance log CSV covering all metals and all timesteps, plus diagnostic emission maps for each metal. These outputs feed directly into the inverse modelling pipeline: the mass values at any observation cell and timestep form the observation vector d used by the NNLS bifurcation solver.

2. Mathematical foundation

2.1 The governing equation

Within each river channel segment the model solves the one-dimensional advection–diffusion–decay equation:

$$\partial C/\partial t + u \cdot \partial C/\partial x = Ex \cdot \partial^2 C/\partial x^2 - K \cdot C + S(x,t)$$

Symbol	Units	Meaning
C(x,t)	mg/m³	Pollutant concentration at position x and time t
u	m/s	Mean flow velocity (positive = downstream)
Ex	m²/s	Longitudinal dispersion coefficient — physical spreading
K	1/s	First-order decay / retardation coefficient
S(x,t)	mg/(m·s)	Source term — pollutant emission rate per unit channel length
x	m	Distance along channel from upstream boundary
t	s	Time

2.2 Eigenfunction series solution

Following Zhang & Xin (2017), for a finite domain x ∈ [0, L] with homogeneous boundary conditions (zero-flux at both ends), the concentration due to a continuous point source of strength M_src at position x_src, beginning at t = 0, is:

$$C(x,t) = \Sigma_n (2/L) \cdot [M_{src} \cdot \sin(n\pi \cdot x_{src}/L)] \cdot [(1 - \exp(-\beta_n \cdot t)) / \beta_n] \cdot \sin(n\pi x/L)$$

where the eigenvalue β_n combines dispersion and decay:

$$\beta_n = Ex \cdot (n\pi/L)^2 + K$$

The series is summed for n = 1, 2, …, n_max. Convergence is rapid for smooth plumes (n_max = 100 is sufficient) and slower for sharp concentration fronts (n_max = 500+). Advection is applied separately by shifting the concentration profile by u·t via interpolation, which is exact for uniform velocity.

2.3 Finite release window — superposition

Sources release pollutants only during the window [t_start, t_end]. By linear superposition of two continuous solutions started at different times:

$$C(x,t) = C_{continuous}(x, t - t_{start}) - C_{continuous}(x, t - t_{end})$$

The second term is subtracted only when t > t_end. This formulation is exact — no approximation is introduced by the finite release window.

2.4 Gaussian transport operator (PCRaster integration)

Applying the full eigenfunction series to every cell of a PCRaster raster at every timestep would be prohibitively slow. Instead, the model uses the equivalent Gaussian kernel, which is the exact Green's function of the ADE for uniform u and Ex over a single timestep ΔT:

$$M_{new}(x) = \int M_{old}(x') \cdot G(x - x' - u \cdot \Delta T,\ \sigma^2) \cdot \exp(-K \cdot \Delta T)\, dx'$$

where G is a normalised Gaussian kernel with variance:

$$\sigma^2 = 2 \cdot Ex \cdot \Delta T$$

In practice each source cell i is treated as a Dirac delta with strength M_i. Its mass advects by u·ΔT and spreads into a Gaussian of width σ. The contributions from all cells are summed to give M_new. This is mathematically identical to the Method of Characteristics with analytical dispersion, and it is mass-conservative by construction (the kernel is normalised).

2.5 Why this eliminates numerical diffusion

The original first-order upwind scheme introduces a truncation error that acts as an artificial diffusion with effective coefficient:

$$Ex_{numerical} = u \cdot \Delta X / 2 = 1.8 \times 100 / 2 = 90\ m^2/s$$

This is independent of the physical Ex and is purely a grid artefact. Typical physical dispersion coefficients in rivers are 1–50 m²/s — an order of magnitude smaller. The Gaussian operator uses only the specified physical Ex (default 10 m²/s), so plumes are roughly nine times sharper than the FVM output for the same grid size.

Key result: Setting Ex = 10 m²/s in the analytical model produces the same total mass as the FVM, but plumes are sharper and physically correct. The FVM was effectively simulating Ex = 90 m²/s regardless of what value was intended.

3. Multi-metal formulation

3.1 Superposition of independent metal transports

Because the governing equation is linear and heavy metals are treated as conservative (no cross-species reactions), each metal can be transported independently. The total pollutant field is decomposed by species:

```
M_total(x,t)  =  Σ_m  M_m(x,t)      for metals m ∈ {Cd, Cr, Pb, Cu, Zn, …}
```

Each metal m has its own emission map E_m, mass map M_m, and concentration map C_m. All metals share the same discharge field Q(x,t) and LDD network. This means one set of PCRaster maps is loaded per timestep and reused for all metals, keeping I/O efficient.

3.2 Signature decomposition

Each source cell k has a total mass release M_k [mg] and a chemical signature vector s_k = [f_Cd, f_Cr, f_Pb, f_Cu, f_Zn] where the fractions f_m sum to 1.0. The per-metal emission rate at source cell k is:

```
E_m(k)  =  M_k · f_m(k) / ΔX      [mg/m]
```

This decomposition preserves the total mass: summing E_m(k) over all metals m recovers M_k / ΔX. The signature fractions are loaded from the signature CSV and auto-normalised if they do not sum exactly to 1.0.

3.3 Observation vector for NNLS

At any observation cell (row, col) and timestep t_obs, the per-metal mass values form the observation vector required by the NNLS inverse solver:

```
d  =  [M_Cd(obs, t_obs),  M_Cr(obs, t_obs),  M_Pb(obs, t_obs),  M_Cu(obs, t_obs),
M_Zn(obs, t_obs)]
```

This vector is read directly from the per-metal output maps. The NNLS system d = S·x then solves for the source contribution vector x.

4. Code architecture

The script is organised into six functional layers, each with a clear responsibility. The table below lists every function and class in order of execution.

Component	Type	Responsibility
_safe_float()	function	Safely converts a CSV cell string to float. Handles empty strings, 'NA', 'nan', whitespace, and BOM characters without raising exceptions.
load_signature_csv()	function	Reads source_signatures.csv. Validates and normalises metal fractions. Skips blank rows, zero-mass sources, and rows with unparseable coordinates. Returns list of source dicts and list of metal names.
build_metal_emission_maps()	function	Converts the source list into one PCRaster scalar map per metal. Each map holds E_m [mg/m] at source cell locations. Saves diagnostic maps to source_metal_maps/.
_analytical_concentration()	function	Core eigenfunction series solver. Computes C(x,t) at arbitrary positions for a single continuous point source. Applies advection shift by interpolation. Returns np.ndarray.
analytical_step()	function	Advances a 1-D concentration array by ΔT using the Gaussian kernel. Treats each cell as a Dirac source. Used by AnalyticalModel1D.
apply_analytical_operator_pcr()	function	PCRaster interface for the Gaussian operator. Converts M map to NumPy, applies cell-wise advect+spread, converts back to PCRaster. Returns updated M map.
AnalyticalMultiMetalModel	class (DynamicModel)	Main PCRaster dynamic model. initial() loads all data and builds emission maps. dynamic() runs the transport loop: analytical operator → confluence routing → source injection → mass balance → file output.
AnalyticalModel1D	class	Standalone single-channel analytical model requiring no PCRaster. Used for validation, sensitivity testing, and per-segment sub-modelling in the inverse pipeline.
compare_fvm_vs_analytical()	function	Runs both FVM upwind and analytical solutions on a 1-D test channel for direct comparison. Returns concentration arrays and numerical diffusion estimate.

4.1 Execution flow — basin mode

1. Script starts: CONFIG is parsed, DynamicFramework is created.
2. initial() is called once: LDD map loaded, signature CSV parsed, per-metal E maps built, state dicts initialised, mass balance log created.
3. For each timestep t = 1 to nrOfTimeSteps, dynamic() is called:
   a. Q map for timestep t is read from disk (readmap).

b. For each metal m:
    i. Source emission E_m applied if $t \in [release\_start, release\_end]$.
    ii. apply_analytical_operator_pcr(): Gaussian advect+spread on M_m.
    iii. FluxOut = $Q \times C_m$ computed; upstream() merges at confluences.
    iv. $M_{new} = M_{transported} - (\Delta T/\Delta X) \times (FluxOut - FluxIn) + \Delta T \times E_m$.
    v. Positivity enforced; C_m updated.
    vi. Ocean cell mass tracked; mass balance computed and logged.
    vii. M_m and optionally C_m written to disk via pcr.report().
4. Simulation ends; log and output paths printed.

5. Detailed function and class reference

5.1 load_signature_csv(csv_path)

Item	Detail
Purpose	Parse source_signatures.csv into a list of source dicts and extract metal names from the header.
Returns	sources: list of dicts | metals: list of str
BOM handling	Field names are stripped of the UTF-8 BOM (\ufeff) that Excel adds when saving CSVs — a common silent failure source.
Empty cells	Any metal cell that is blank, 'NA', 'nan', 'N/A', 'none', or '-' is converted to 0.0 without raising an error.
Normalisation	Fractions summing to ≠ 1.0 are auto-normalised. If the sum > 2.0 the warning specifically states 'looks like raw mg values' to flag a likely data entry error.
Skip conditions	Rows skipped with warning: blank rows, total_mass ≤ 0, all metal values = 0, unparseable row/col/total_mass.
Fatal error	Raised if no valid sources remain after filtering, or if no metal columns are found in the header.

5.2 build_metal_emission_maps(sources, metals, base_poll_map, deltaX, output_dir)

Item	Detail
Purpose	Create one PCRaster scalar map per metal containing E_m [mg/m] at each source cell.
Formula	E_m[r,c] = total_mass × fraction_m / deltaX for each source at (row, col).
Additive	Multiple sources at the same cell are accumulated (+=), not overwritten.
Grid check	Source cells outside the raster dimensions are skipped with a warning.
Diagnostic	Each map is saved as E_nominal_<metal> in source_metal_maps/ for visual inspection.
Returns	Dict { metal_name : PCRaster scalar map }

5.3 _analytical_concentration(x_arr, t, x_src, M_src, L, u, Ex, K, n_max)

Item	Detail
Purpose	Evaluate the eigenfunction series solution at positions x_arr, time t, for a single continuous source.
Formula	$C(x,t) = \Sigma_n (2/L) \cdot M_{src} \cdot \sin(n\pi x_{src}/L) \cdot [(1-\exp(-\beta_n \cdot t))/\beta_n] \cdot \sin(n\pi x/L)$

β_n	Ex·(nπ/L)² + K — combines dispersion and decay eigenvalues.
β_n = 0	Limit handled analytically: (1−exp(−β·t))/β → t as β → 0, avoiding division by zero.
Advection	Applied separately: x_shifted = x − u·t, then np.interp() remaps C onto shifted grid. Exact for uniform u.
Positivity	np.maximum(C, 0) enforced — series can overshoot to small negatives near boundaries.
Returns	np.ndarray of shape (len(x_arr),)

5.4 apply_analytical_operator_pcr(M_map, Q_map, ldd, velocity, Ex, K, deltaT, deltaX, n_max)

Item	Detail
Purpose	Apply the Gaussian dispersion+advection operator to a PCRaster mass map for one timestep.
Step 1	M_map and Q_map converted to NumPy arrays via pcr2numpy().
Kernel width	half_w = ceil(3σ/deltaX) — the kernel spans ±3σ around the advected centre (3σ rule captures 99.7% of mass).
Advection	adv_cells = u·ΔT/ΔX. Fractional cell destination split linearly between floor and ceil cell.
Dispersion	Gaussian kernel normalised to sum = 1.0 ensures exact mass conservation.
Decay	Each cell multiplied by exp(−K·ΔT). For K=0 this is 1.0 — no overhead.
pcr.report vs self.report	pcr.report() is used for output because self.report() (DynamicModel framework) rejects filenames containing a dot, which our timestep-numbered filenames require.
Returns	PCRaster scalar map of updated mass M_new [mg/m].

5.5 AnalyticalMultiMetalModel (class, inherits DynamicModel)

initial()

Called once before the time loop. Loads and validates all inputs, builds per-metal emission maps, creates output directories, initialises state dictionaries (M, C, ocean_release, Theoretical_Total) for every metal, and writes the mass balance log header.

_apply_release(metal, t)

Returns E_nominal[metal] (the emission map) if t is within [release_start, release_end], or a zero scalar map otherwise. This is evaluated per-metal per-timestep.

dynamic()

The main time-stepping method. Called once per timestep by DynamicFramework. Reads Q once per timestep, then loops over all metals applying the analytical transport operator, PCRaster upstream routing, source injection, mass balance accounting, and file output. Console output is printed every 50 timesteps and for the first 5 steps.

_map_path(metal, step, prefix)

Generates the output filepath for a given metal and timestep. Uses PCRaster block/slice naming: block = (step−1) // 999, slice = (step−1) % 999 + 1. Example: metal='Cd', step=1000 → 'Cd/M_Cd0000001.001'.

5.6 AnalyticalModel1D (class, no PCRaster dependency)

Method	Purpose
__init__(L, u, Ex, K, n_max, deltaX)	Initialise with channel parameters. Parameters match AdvecDiffu.py exactly.
run(source_positions, source_masses, release_windows, x_obs, time_values)	Run single-metal forward simulation. Returns C_matrix of shape (n_times × n_x). Uses superposition over the release window.
run_multimetal(source_positions, source_total_masses, release_windows, signature_matrix, metal_names, x_obs, time_values)	Decomposes total masses by metal using signature_matrix (shape n_metals × n_sources), calls run() independently for each metal. Returns dict { metal : C_matrix }.

6. Inputs

6.1 PCRaster map files

CONFIG key	File type	Description
clone_map	.map (PCRaster)	Spatial reference: grid dimensions, cell size, projection, NoData pattern. All other maps must be cloned from this.
ldd_map	.map (LDD type)	Local drain direction. Encodes full river network topology. Each cell = integer 1–9 indicating drain direction. Must be sound (no unsound cells).
pollutant_map	.map (Scalar)	Used only for its grid structure (array shape and NoData mask). Its numeric values are entirely replaced by the signature CSV.
flow_map	.map time series	Discharge Q [m³/s] maps, one per timestep. Path prefix only — the framework appends block/slice suffix automatically.

6.2 source_signatures.csv — format specification

This is the key new input. Each row describes one pollutant source cell with its grid location, total mass released, and the chemical composition breakdown across metal species.

Column	Type	Units	Constraints	Description
source_id	string	—	unique within file	Label for this source. Used in log output only.
row	integer	—	1-based, within grid	PCRaster row of source cell (row 1 = top row).
col	integer	—	1-based, within grid	PCRaster column of source cell (col 1 = leftmost).
total_mass	float	mg	> 0	Total pollutant mass released during [release_start, release_end].
Cd	float	fraction	0–1, sums to 1	Cadmium fraction of total_mass. Column name is the metal label used in output directories.
Cr	float	fraction	0–1, sums to 1	Chromium fraction.
Pb	float	fraction	0–1, sums to 1	Lead fraction.
Cu	float	fraction	0–1, sums to 1	Copper fraction.
Zn	float	fraction	0–1, sums to 1	Zinc fraction. Additional metals (Al, Fe, Ni, …) added as extra columns — no code change required.

Important: Fraction columns must sum to 1.0 per row. The loader auto-normalises with a warning if they do not. If the sum is >> 1 (e.g. 344), this indicates raw mg values were entered instead of fractions — the loader normalises automatically but prints a clear warning. Correct the CSV before final runs.

Example CSV content

```
source_id,row,col,total_mass,Cd,Cr,Pb,Cu,Zn
SRC_01,45,12,500.0,0.16,0.21,0.18,0.11,0.34
SRC_02,60,30,800.0,0.16,0.34,0.25,0.24,0.01
SRC_03,80,55,300.0,0.32,0.24,0.01,0.26,0.17
SRC_04,100,70,650.0,0.23,0.13,0.29,0.18,0.17
```

6.3 CONFIG parameters

Parameter	Default	Units	Description
output_dir	—	path	Root output directory. Subdirectories per metal created automatically.
deltaT	1	s	Timestep. CFL: u·ΔT/ΔX ≤ 1 (default: 1.8×1/100 = 0.018 ✓).
deltaX	100	m	Cell size. Determines spatial resolution of all maps.
velocity	1.8	m/s	Mean flow velocity. Used for A = Q/velocity and as fallback.
release_start	1	timestep	First timestep of pollutant emission.
release_end	2	timestep	Last timestep of pollutant emission.
nrOfTimeSteps	2000	—	Total simulation timesteps. Duration = nrOfTimeSteps × deltaT.
ocean_cell	(278,18)	(row,col)	1-based outlet cell. Tracks cumulative mass leaving domain.
Ex	10.0	m²/s	Physical longitudinal dispersion. Set < 90 for sharper plumes than FVM. Typical rivers: 1–100 m²/s.
K	0.0	1/s	First-order decay. Set 0 for conservative heavy metals.
n_max	100	—	Eigenfunction series terms. 100 for smooth plumes; 500+ for sharp fronts.
segment_length	5000	m	Domain length L for eigenfunction boundary conditions. Set to longest channel segment length.
report_concentration	True	—	If True, saves C [mg/m³] maps alongside M [mg/m] maps. Doubles output volume.

7. Output files

File / pattern	Description
<output_dir>/<METAL>/M_<METAL>XXXXXXX.XXX	PCRaster map of mass per unit length M [mg/m] for metal METAL at each timestep. Block/slice naming: step 1 → M_Cd0000000.001; step 1000 → M_Cd0000001.001.
<output_dir>/<METAL>/C_<METAL>XXXXXXX.XXX	Concentration map C [mg/m³]. Only written when report_concentration = True.
<output_dir>/mass_balance_log.csv	One row per (timestep, metal). Columns: timestep, metal, Theoretical_Total [mg], Model_Total [mg], Error [mg], ErrorPct [%]. Used to verify mass conservation.
<output_dir>/source_metal_maps/E_nominal_<METAL>	Diagnostic: static emission rate map E_m [mg/m] for each metal. Load in QGIS or ArcGIS to verify source locations and relative magnitudes.

7.1 Mass balance log interpretation

Column	Meaning
Theoretical_Total	Cumulative mass injected = Σ_t maptotal(E_m × ΔX) × ΔT [mg]. Grows only during [release_start, release_end].
Model_Total	Mass currently in domain = maptotal(M_m) × ΔX + ocean_release [mg]. Should track Theoretical_Total closely.
Error	Theoretical_Total − Model_Total. Should remain near zero. Persistent growth indicates a bug or numerical instability.
ErrorPct	Error as percentage of Theoretical_Total. Values < 1% indicate acceptable mass conservation.

8. Known limitations and future work

Limitation	Impact	Suggested fix
Gaussian operator uses column-direction advection	In a 2-D raster where channels flow at angles to the grid columns, the advection component of the Gaussian operator is approximate. PCRaster upstream() corrects the network routing, but within-cell mass distribution may be slightly off.	Extract true 1-D flow paths from the LDD and apply the analytical solution along each path. Computationally more expensive but exact.
Uniform velocity assumption in apply_analytical_operator_pcr()	The function contains a line that overwrites per-cell velocity with the global CONFIG velocity. This simplification is intentional for now but discards spatial Q variation within the Gaussian step.	Remove the override line and use Q_arr / (Q_arr / velocity + 1e-30) per cell for spatially variable velocity.
Fixed release window	All sources start and stop emitting at the same timesteps (release_start, release_end). Real pollution events have source-specific timing.	Add t_start and t_end columns to the signature CSV and apply per-source release windows.
Constant Ex and K	Dispersion and decay are spatially uniform. Real rivers have reach-dependent Ex (function of channel width and depth).	Add Ex and K columns to the signature CSV or define reach-specific maps.
Conservative pollutants only	Adsorption, sedimentation, and pH-dependent speciation are not modelled. Acceptable for dissolved heavy metals in short simulation windows.	Add a retardation factor R and adsorption term; requires additional field data.
n_max convergence	For very sharp concentration fronts (pulse released in 1–2 timesteps) n_max = 100 may underestimate peak concentrations near the source.	Increase n_max to 500–1000 for pulse inputs. Runtime scales linearly with n_max.

9. Run modes

Command	Mode	Requirements	Output
python ADE_Analytical_Multimetal.py	basin	All PCRaster maps + signature CSV + discharge time series	Per-metal mass maps, concentration maps, mass_balance_log.csv
python ADE_Analytical_Multimetal.py --mode validate	validate	No map files needed	Console comparison + validation_fvm_vs_analytical.png
python ADE_Analytical_Multimetal.py --mode 1d	1d	No map files needed	multimetal_analytical_1d.png — per-metal concentration profiles for built-in 2-source example

10. Pre-run checklist

- clone_map path is correct and the file is a valid PCRaster map.
- ldd_map is sound (no unsound cells) and covers the same extent as clone_map.
- flow_map prefix resolves to an existing file when the suffix Q0000000.001 is appended.
- signature_csv exists; open it and verify: (a) fraction columns present, (b) no blank header cells, (c) fractions roughly sum to 1 per row.
- release_start and release_end are both within [1, nrOfTimeSteps].
- CFL condition satisfied: velocity × deltaT / deltaX ≤ 1.0 (default: 1.8 × 1 / 100 = 0.018 ✓).
- ocean_cell (row, col) is within grid bounds and on a channel cell.
- output_dir has write permission (the script creates it if missing).
- If running in Spyder / IPython with %runfile, check that the working directory is set to the script folder.

11. References

Zhang, H. & Xin, P. (2017). *Analytical solution of 1-D advection–diffusion equation.* Used as the basis for the eigenfunction series solution implemented in _analytical_concentration().

Karssenberg, D. et al. (2010). *A software framework for construction of process-based stochastic spatio-temporal models and data assimilation.* Environmental Modelling & Software, 25, 489–502. PCRaster framework documentation.

Moghaddam, M.B. et al. (2021). *Inverse modeling of contaminant transport for pollution source identification in surface and groundwaters: a review.* Groundwater for Sustainable Development, 100651.